

# Functional programming and control

**Block 1** — control patterns: recursion, the stack, fold, evaluation

---

SW7 · Aalborg University Copenhagen · Autumn 2026

# Programs = Algorithms + Data Structures (Wirth, 1976)

The two previous sessions were about data structures:

S2            how data are **modified**: persistent and ephemeral data structures

S3            how data are **constructed**: algebraic data types and objects

S4, S5        paradigms of how algorithms run

Sessions 4 to 9 are about one important aspect of the algorithms part — **the control flow of the execution** — and the design space of **who** and **what** makes the next step happen.

1. when a function is **called**: its arguments, its frame, its return
2. across an **iteration**: a loop, a collection, a stream
3. when a computation is **suspended** and resumed: callbacks, promises, generators
4. when several computations run **at once**: threads, processes, a parallel fold

# Today

## Block 1 (Implicit/Explicit Control)

Who, and how, decides the next step of a computation? Four parts, four questions:

1. **What is computing?**
2. **What does a machine need to call a function?**
3. **Do we need a loop/recursion as syntax construct to run loops?**
4. **In which order, and when, are arguments evaluated?**

## Block 2 ( $\lambda$ -calculus)

The  $\lambda$ -calculus: the smallest language that computes everything, its evaluation strategies, and booleans, numbers, `if` and recursion built from three forms.

### Lab

An evaluator for the calculus, in OCaml: terms as a type, substitution, two strategies, closures. The same program grows a type checker in session 10.

# Two definitions of computing

1936, before any computer

# What is *computing*?

Two answers, both from 1936:

a processor executing instructions and changing the state, step-by-step, using a transition function

**Turing**

a calculus transforming a program, according to a rewriting relation, to evaluate a function applied to its arguments

**Church**

**imperative/functional**  
style

a global state, changed by assignment and sequencing (von Neumann 1945)

expressions and application, no state (Lisp 1958, ML 1973)

**machine/calculus**  
model

an operational model of *how* — the tool of complexity theory

an equational model of *what* — the foundation of language semantics (Landin 1964, Scott 1970, Plotkin 1975)

**explicit/implicit** control

the order of the steps is written: ";", loops, the program counter

the order is not written: a **strategy** chooses — this session

# Turing, 1936: a machine

- a finite set of **states**
- a **tape** of cells, unbounded, one symbol per cell
- a **head** that reads one cell
- one **rule** per state and symbol: write, move, next state

| state | reads | writes | moves | next |
|-------|-------|--------|-------|------|
| go    | 1     | 1      | →     | go   |
| go    | ␣     | 1      | ←     | back |
| back  | 1     | 1      | ←     | back |
| back  | ␣     | ␣      | →     | halt |

A machine that adds one to a number written in unary.



Control is a **position**: the state and the head. The next step is a **function** of them: exactly one move.

# Church, 1936: a calculus

Terms are built from three forms:

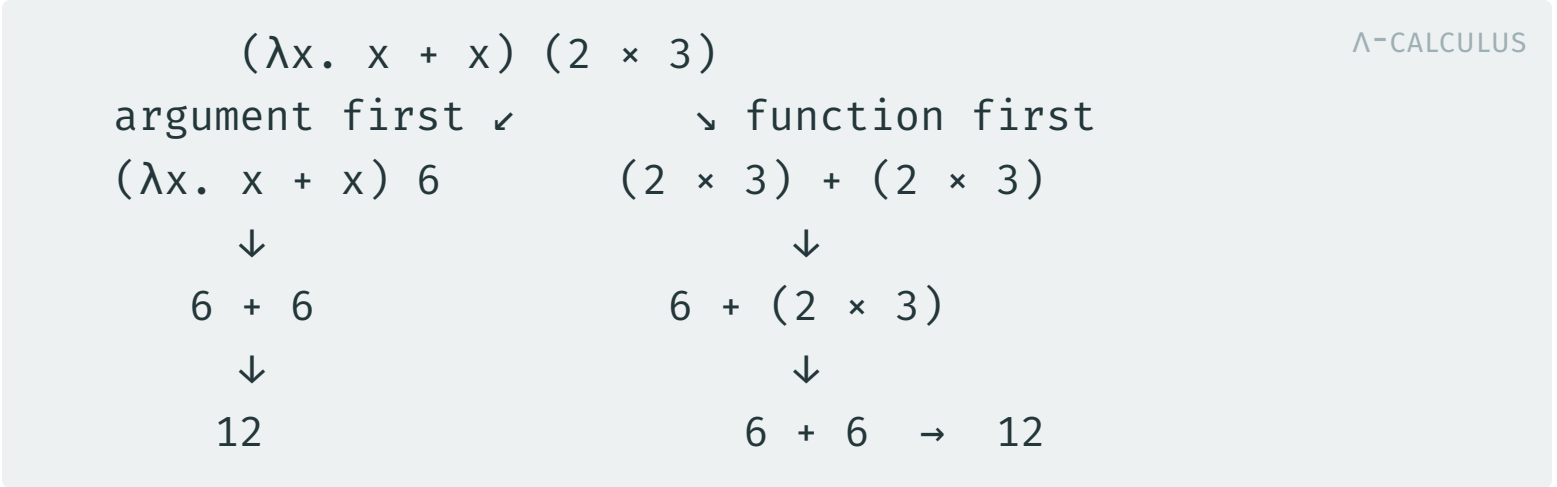
- a **variable** — `x`
- a **function** — `λx. t`
- an **application** — `t u`

```
0 (λx. x + x) (2 × 3)
```

Step Run Reset

☒ argument first ☐ function first

One rule: an application is replaced by the function's body, with the argument put in for the variable. A term can offer two places to apply it:



Numbers, `+` and `×` are not part of the calculus; they keep this first example readable. Block 2 defines numbers from the three forms.

Control is a **choice**: a step is allowed at any application; nothing in the term picks one. The next step is a **relation** — a **strategy** makes it a function (block 2).

# Different answers, same expressive power

- Turing, 1937, in an appendix: every function the lambda-calculus defines, a machine computes, and the other way round.
- Kleene, 1936: a third definition, *recursive functions* — the same functions again.
- Every model of effective computation proposed since has turned out to compute the same functions. This is the **Church–Turing thesis**.

Equal in what they can compute. Different in one design decision: **who decides the next step**.

# The two definitions, as machines and as languages

|                    | the machine                                                             | the calculus                              |
|--------------------|-------------------------------------------------------------------------|-------------------------------------------|
| built, 1945 / 1958 | the stored-program computer (von Neumann)                               | Lisp (McCarthy)                           |
| control            | a program counter and a stack                                           | substitution + a choice of what to reduce |
| constructs         | <code>;</code> <code>while</code> <code>x := e</code> (and many others) | just function definition and application  |
| the default in     | C, Java, Python, JavaScript                                             | Haskell, OCaml, Scheme, Erlang            |

Paradigm  $\neq$  language: Java has lambdas, OCaml has `for`. A language makes one of the two the default.

# Recursion and the stack

What does a machine need to call a function?

A call also needs an order and a passing mode for its arguments — part 4, separately.

# A call, on the machine

```
int sum(int *a, int n) {  
    if (n == 0) return 0;  
    return a[0] + sum(a + 1, n - 1);  
}
```

c

`sum` calls itself before it can add. The machine must remember, for every call not yet finished three things:

1. where to continue — the **return address**
2. the arguments — `a`, `n`
3. what is left to do — `3 + _`

One such record is a **frame**. Frames are pushed at a call and popped at a return: the **call stack**.

The machine does not substitute. It **stores** the argument in the frame, and what remains to be done on the stack.

# The same call, in the calculus

the machine: the stack

```
sum [] returns 0
```

```
sum [4] pending 4 + _
```

```
sum [1; 4] pending 1 + _
```

```
sum [3; 1; 4] pending 3 + _
```

Step

Run

Reset

the base case: `sum []` is 0

The stack holds what the calculus keeps *in the term*: the surrounding computation. Same information, two places.

the calculus: the term

```
3 + (1 + (4 + (sum [])))
```

# When the stack runs out

```
def sum_list(xs):  
    if not xs:  
        return 0  
    return xs[0] + sum_list(xs[1:])  
  
print(sum_list(list(range(100))))  
print(sum_list(list(range(10_000))))
```

PYTHON

```
4950  
Traceback (most recent call last):  
  File "recursion_limit.py", line 4, in sum_list  
    return xs[0] + sum_list(xs[1:])  
  [Previous line repeated 996 more times]  
RecursionError: maximum recursion depth exceeded
```

recorded

## language the stack ends at\*

|         |                                                          |
|---------|----------------------------------------------------------|
| Python  | 1 000 frames, by a counter — a deliberate choice         |
| Java    | <code>StackOverflowError</code> after 33 436 frames here |
| C       | 8 MB ( <code>ulimit -s</code> ): a segmentation fault    |
| OCaml 5 | the stack lives on the heap and grows: 1 GB              |

\*One measurement; concrete numbers might differ.

A recursion that is correct in the calculus can fail on the machine: the stack is finite, the term is not.

# Can a loop replace every recursion?

```
deep = ("Note", "C4") # session 3's melody tree, as tuples
for i in range(100_000): deep = ("Seq", ("Note", "D4"), deep)
```

PYTHON

```
def count_rec(m):
    if m[0] == "Note": return 1
    return count_rec(m[1]) + count_rec(m[2])
print(count_rec(deep))
```

PYTHON

Run

```
File "<exec>", line 5, in count_rec
File "<exec>", line 5, in count_rec
[Previous line repeated 992 more times]
RecursionError: maximum recursion depth exceeded
```

live

```
def count(m):
    n, stack = 0, [m]
    while stack:
        m = stack.pop()
        if m[0] == "Note": n += 1
        else: stack += [m[2], m[1]]
    return n
print(count(deep))
```

PYTHON

Run

100001

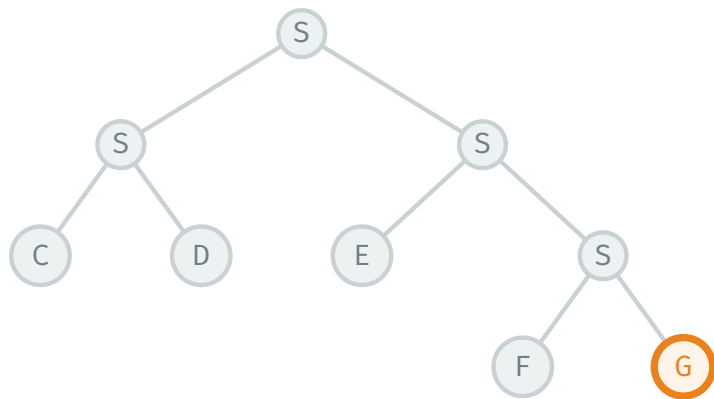
live

A tree 100 000 notes deep. Right: session 2's stack in a new job — Lab 2's maze visit was this loop.

# What the explicit stack holds

```
def count(m):  
    n, stack = 0, [m]  
    while stack:  
        m = stack.pop()  
        if m[0] == "Note": n += 1  
        else: stack += [m[2], m[1]]  
    return n
```

PYTHON



the stack

*empty*

count = 5

Step

Run

Reset

pop a Note: count one

Each entry is *what to do next* once the current subtree is counted — the rest of the computation. Session 5 names it: a **continuation**. Sessions 6 and 7 hand it to a scheduler: a thread is a stack the operating system saves; a coroutine is a suspended computation the program saves itself.

# Does a tail call need the stack?

```
let rec sum acc = function
  | [] -> acc
  | x :: xs -> sum (acc + x) xs

sum 0 [3; 1; 4]      (* 8 *)
```

OCaml

After the call, nothing is left to do: no pending `3 + _`. The accumulator carries what the stack would have held.

```
let sum xs =
  let acc = ref 0 and l = ref xs in
  while !l <> [] do
    acc := !acc + List.hd !l;
    l := List.tl !l
  done; !acc
```

OCaml

The compiled code of the left *is* the right: the frame is reused, the call becomes a jump.

\* A **tail call** is a call in the position where its result is returned unchanged.

# Proper tail calls are a language decision

## guaranteed

Scheme — central to the original 1975 design; required by every standard since

Haskell, Erlang — iteration is normally written this way; OCaml optimises it, and also has `for` and `while`

Lua, Elixir

## not guaranteed

Python — refused on purpose: the traceback would lose frames

Java — the JVM has no tail-call instruction

C — an optimisation the compiler may or may not apply

Whether a tail call needs a frame is decided by the language, not by the program. Same program, two behaviours.

# Recursion and loops, stated

`loop` → `recursion` Every loop can be written as a tail-recursive function: the loop variables become its arguments.

`recursion` → `loop` Every recursion can be written as a loop with an explicit stack: the stack holds the pending work.

Equal power; the difference is what the program writes down and what the language keeps hidden. Compilers perform both translations: a tail call becomes a jump, a recursion becomes a loop over an explicit stack — a stack of **continuations**, session 5.

Sources: recursion elimination — Bird 1977; the construction — Reynolds 1972; a tail call as a jump — Steele 1977.

# Implicit control: the loop as a value

Do we need a loop construct at all?

# Observation: one recursion, three bodies

```
let rec sum = function  
  | [] -> 0  
  | x :: xs -> x + sum xs
```

OCAML

```
let rec product = function  
  | [] -> 1  
  | x :: xs -> x * product xs
```

OCAML

```
let rec length = function  
  | [] -> 0  
  | x :: xs -> 1 + length xs
```

OCAML

Three functions, one recursion. What differs: the value for `[]`, and how `x` is combined with the result for the rest.

If the two differences were *arguments*, one function would do for all three.

# The body as a parameter: fold

OCaml

```
let rec fold f acc = function
  | [] -> acc
  | x :: xs -> fold f (f acc x) xs

let sum      = fold (fun acc x -> acc + x) 0
let length = fold (fun acc _ -> acc + 1) 0
```

# Two folds: the order in which `f` is applied

```
let rec fold_left f acc = function
  | [] -> acc
  | x :: xs -> fold_left f (f acc x) xs
```

OCaml

`f` runs **as soon as** an element is met, from the left, and its result is carried along as `acc`.

The recursive call is the last thing done — a **tail call**, like `sum acc` on *Does a tail call need the stack?*. The frame is reused: a loop, no stack.

```
let rec fold_right f xs z = match xs with
  | [] -> z
  | x :: xs -> f x (fold_right f xs z)
```

OCaml

`f` runs **only after** the rest of the list is done, so the first application to run is the last element's — from the right.

The recursive call is *inside* `f x _`: not a tail call. One pending frame per element — the stack of *The same call, in the calculus*.

Two orders of applying the same `f`: left to right with a carried value, or right to left with pending work.

# Same f, same list — two answers?

With `f = (+)`, `z = 0`:

```
fold_left:  ((0 + 3) + 1) + 4 = 8
fold_right: 3 + (1 + (4 + 0)) = 8
```

OCaml

The same result: addition does not care where the parentheses go.

With `f = (-)`, `z = 0`:

```
fold_left:  ((0 - 3) - 1) - 4 = -8
fold_right: 3 - (1 - (4 - 0)) = 6
```

OCaml

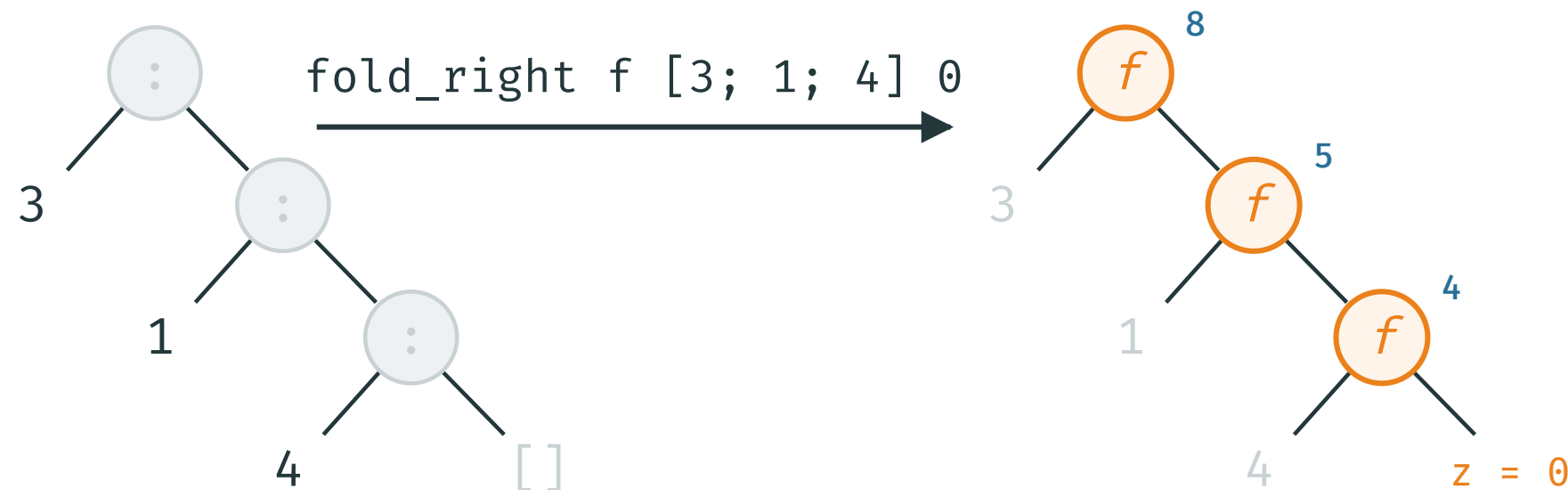
Two results: subtraction does care.

The order of applying `f` is part of the meaning — unless `f` is **associative** and the seed is its **identity**: then only the parenthesisation changes. Session 9 makes that the condition for running a fold in parallel.

**Algorithm = Logic + Control** (Kowalski, 1979), read off the code: `f` and `z` are the logic, `fold_left` or `fold_right` the control. With `+` the control changes only the cost; with `-` it changes the answer.

# fold as a rewriting of the list's tree

A list is a tree of `(:)` nodes ending in `[]`. `fold_right f xs z` replaces every `(:)` by a call to `f` and `[]` by `z` — same shape; `fold_left f z xs` builds the mirrored tree from `z` upwards.



Step

Run

Reset

fold\_right

(+)

fold\_left

f = (-)

f =

result 8 at the root — the last f to run was the first written

# The same loop in four languages

| language   | the sum of a list                                  | and its friends                                         |
|------------|----------------------------------------------------|---------------------------------------------------------|
| OCaml      | <code>List.fold_left (+) 0 xs</code>               | <code>List.map</code> , <code>List.filter</code>        |
| Python     | <code>functools.reduce(operator.add, xs, 0)</code> | <code>map</code> , <code>filter</code> , comprehensions |
| JavaScript | <code>xs.reduce((acc, x) =&gt; acc + x, 0)</code>  | <code>.map</code> , <code>.filter</code>                |
| Java       | <code>xs.stream().reduce(0, Integer::sum)</code>   | <code>.map</code> , <code>.filter</code>                |

`map` applies a function to every element; `filter` keeps the elements a function accepts. Both are folds.

Session 5: who drives this loop — the collection or the caller — and what else a loop can be.

# Evaluation order and parameter passing

In which order, and when, are arguments evaluated?

# The order of arguments is **not** part in the program

```
let f a b = a + b;;  
f (print_string "first "; 10)  
  (print_string "second "; 20);;
```

OCAML

```
second first - : int = 30
```

recorded

OCaml: right to left — and the manual says *unspecified*.

```
const f = (a, b) => a + b;  
const trace = [];  
const r = f((trace.push('first'), 10),  
            (trace.push('second'), 20));
```

JAVASCRIPT

Run

```
first second → 30
```

JavaScript: left to right, by the standard. C: unspecified.

Never the program. Fixed by the language definition (Java, C#, JavaScript, Python: left to right), left unspecified (C, Scheme, OCaml — right to left in practice), or no order at all (Haskell: an argument is evaluated when used).

# Parameter passing: when is an argument evaluated?

A second question, different from the order: whether an argument is evaluated *before* the call at all.

| mode                 | the argument is evaluated           | languages                                                                             |
|----------------------|-------------------------------------|---------------------------------------------------------------------------------------|
| <b>call by value</b> | before the call, once               | C, Java, Python, JavaScript, OCaml, Scheme                                            |
| <b>call by name</b>  | at each use; never if unused        | Algol 60 (its default); Scala, parameters written<br><code>x: =&gt; T</code>          |
| <b>call by need</b>  | at first use, once; never if unused | Haskell (everywhere); R (every argument is a promise);<br>Scala <code>lazy val</code> |

Both questions — in which order, and when — are answered by the language, not by the program. Block 2 derives the three modes from the calculus.

# Can if be a function?

A test of call by value: write `if` as an ordinary function.

```
let my_if c t e = if c then t else e

let rec fact n =
  my_if (n = 0) 1 (n * fact (n - 1))
```

OCAML

Stack overflow during evaluation  
(looping recursion?).

recorded

```
const myIf = (c, t, e) => c ? t : e;
const fact = n =>
  myIf(n === 0, 1, n * fact(n - 1));
```

JAVASCRIPT

Run

RangeError: Maximum call stack size exceeded

Under call-by-value the arguments are evaluated *before* the call — `fact (n - 1)` too, even when `n = 0`, which recurses forever. In such a language `if` cannot be an ordinary function: it must not evaluate its branches before choosing. So `if` is a **keyword**.

Unless the caller keeps the branches from being evaluated — next slide. Or the language decides otherwise — block 2.

# Can if be a function? — with the branches as functions

```
let my_if c t e = if c then t () else e ()
```

OCAML

```
let rec fact n =  
  my_if (n = 0)  
    (fun () -> 1)  
    (fun () -> n * fact (n - 1))
```

```
- : int = 120
```

recorded

```
const my_if = (c, t, e) => c ? t() : e();  
const factorial = n =>  
  my_if(n === 0, () => 1,  
    () => n * factorial(n - 1));
```

JAVASCRIPT

Run

120

A function's body is not evaluated until the function is *called*. Each branch is now a function of no argument; `my_if` calls only the one it chose, and the other is never run.

So `if` can be a function — if the caller delays the branches by hand. Block 2 gives this a name, and shows the language doing it instead.

# Conclusion of block 1

|   | the question                                       | the answer                                                                                                                                                                                                 |
|---|----------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | What is computing?                                 | Two definitions, 1936. In the machine the next step is a <b>function</b> of the state; in the calculus it is a <b>relation</b> , and a strategy makes it a function.                                       |
| 2 | What does a machine need to call a function?       | A frame. The stack holds what the calculus keeps in the term. Every loop can be written as a tail-recursive function, every recursion as a loop with a stack.                                              |
| 3 | Do we need a loop construct at all?                | No. The recursion is written once, in <code>fold</code> ; a use passes the body and writes no control. <code>fold_left</code> and <code>fold_right</code> differ in the order of applying <code>f</code> . |
| 4 | In which order, and when, are arguments evaluated? | The language decides both: the order (fixed or unspecified) and the mode — by value, by name, by need. Under call by value <code>if</code> cannot be a function unless the caller delays the branches.     |

Each time the choice was implicit, it was **functional programming** expressing control through recursion and function values: recursion rather than an explicit stack, `fold` rather than a loop, a function that delays rather than an argument evaluated first. Block 2 formalises these choices in a minimal setting of the functional paradigm: the  **$\lambda$ -calculus**.